



Week 3

✓ A view of operating System Services

- Services
- System Call
- User Interface

✓ Operating System Services

- User interface
- Program Execution
- I/O operations
- File-system manipulation
- Communications
- Error detection
- Resource allocation
- Accounting
- Protection and Security
- Command Line Interface
 - 2 ways CLI can be implement.
 - why implement as external program is more preferable
 - 2 type of commands.
 - Command to check whether the command is built-in or external program

✓ System Calls

- API - System Call - OS Relationship.(what happen when system call function is called)

- Standard C Library Example
- Type of System Calls

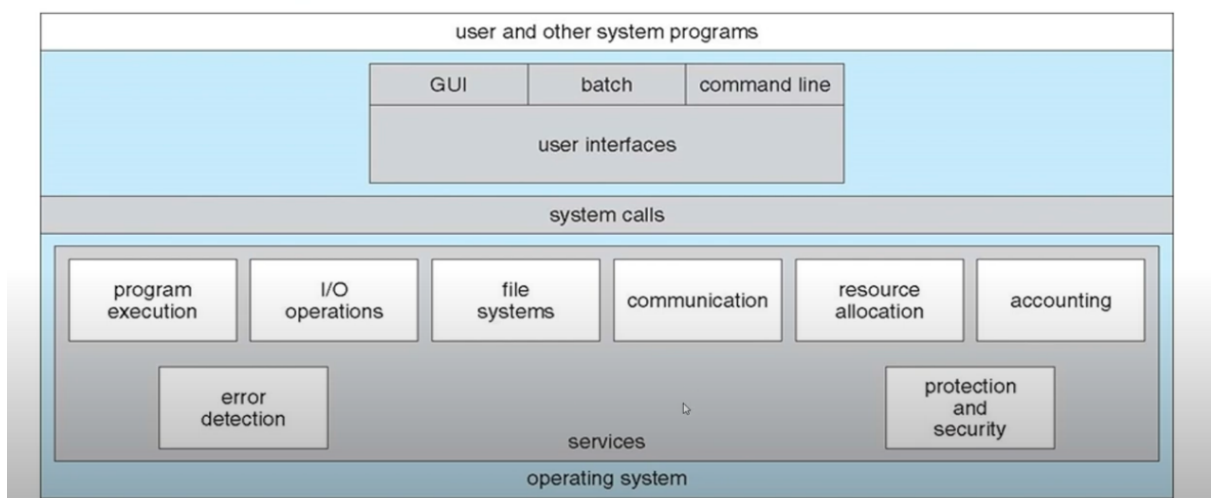
✓ Kernel Architecture

- Monolithic Architecture
- Microkernel Architecture
- Layered Architecture
- Module Architecture (Solaris Example)

| Content

✓ A view of operating System Services

A View of Operating System Services



OS can be divided into 3 major parts

- **Services**
 - the task that os need to do to manage computer resources will be in a from of resources.
 - this service will be call by the program who need to use the resources, but instead of allowing the program to call to the services directly → the program has another level called "system call"

- **📌 System Calls**

- you can think of system calls as program library.
 - so, when a program need services here, the system call the function related to services from system call first.
 - We'll learn the details of system call later.

- **📌 User interfaces**

- It can be GUI, command line, or even batch.
 - batch (group commands into program) - a program, but the program contain the OS command that you have to enter when you use CLI.
 - you may need to do some task many time, and those task you may have to enter 5 commands → you can combine those command into program → so it faster.
 - .bat, .sh

✅ Operating System Services

- **📌 User interface**

- almost all OS have a UI varies between Command-Line (CLI), Graphical User Interface (GUI), Batch

- **📌 Program Execution**

- Service of OS that load a program and run a program for user. Terminate a program as well
- The program can terminate program either normally or abnormally

- **📌 I/O operations**

- when program require I/O, OS need to support this one
- I/O maybe access to file or I/O devices

- **📌 File-system manipulation**

- Normally, program need to read and run file, create directory,....
- Os need to provide this services including permission management
 - this file can only be access by who?
 - the scope of permission to those file (read, write,...)

- **Communications**

- The processes may exchange information
 - What is processes(let's use the term program first)?
- Communication allows 2 or more processes to exchange information
- Ex: when you use line to chat with your friend → you actually don't communicate with your friend → the line program on your machine communicate to the line program on your friend machine
- The communication can occur on the same machine, if 2 processes running on the same machine and need to communicate
- It can also occur to computer over network just like line on your machine and line on your friend machine
- OS provide 2 major way for processes to communicate
 - share-memory
 - message passing
 - (learn later in a class)

- **Error detection**

- OS need to detect error that can occur for both hardware and software, and take the appropriate action to that error
- OS may provide debugging facilities to help user find out what happened

- **Resource allocation**

- when a process need computer resources → OS need to find out the available resources and allocate to the process
- when a process terminate → OS need to deallocate and bring the resource back so that the resources can be allocated for the next process that need them.

- **Accounting**

- OS need to have user information
- So, OS can keep track how much and what kind of resources the user use

- This information can be used in various way → for example, if something went wrong in the system, this info can be used to find out which user cause this problem and what did they do.
- This information can be used to view the user for the system that charge the user for the resources they use.
- **📌 Protection and security**
 - Actually, the 2 term are very closed to each other(protection, security) → sometimes, people think that they are the same → they are not.
 - Protection → ensures that all access to system resources is controlled.
 - The way that we give permission the a user to use the resources.
 - For ex, this user can read only to the file
 - Security → major task: user authentication.
 - The idea is username and password.
- **📌 Command Line Interface (command interpreter, shell)**
 - Why do we need to use command line, GUI is pretty easy?
 - There are many task that can be done only using command line.
 - Some task uuuuu 1,000 .doc files

```
del *.doc
```

2 ways CLI can be implemented

- CLI can be implemented in kernel
- CLI can be implemented as external program (this is is more preferable than in kernel)

Why external program implementation is more preferable?

- The reason is if we implement CLI as part of kernel.
 - CLI is UI → the part that always change
 - Every time we need to modify cli → we modify kernel of OS.
 - we have to shutdown the system, compile the new source code → recompile the kernel again

- As External Program: the new version of CLI is just like upgrade the program.
- We can have more than 1 CLI (external program)
 - Ex, windows we have at least command prompt, power shell
 - unix-based → multiple shell programs.

2 types of commands

- To use CLI, we run the CLI program, then we enter command.
- There are 2 types of command
 - Commands built-in
 - This command are part of the CLI program, when we run the CLI program → this command are loaded into the memory. (အဲ့ဒါက memory ထဲမှာပဲ load ပါနေတာမို့ cli အလွန်)
 - When we enter this command → the command will start to run very fast because it already been loaded into memory.
 - Program
 - Program store on storage → if we use this kind of command, cli ask the OS service to load and run the programs
 - Program must be loaded from storage to memory first
 - Command built-in run faster.



Command built-in, it just like CLI in kernel → because if you want to upgrade the version of command built-in, you have to upgrade entire CLI program. But for command that are external program, if you want to upgrade, the idea is just upgrade the version of program.

Command to check whether the command is built-in or external program

```
sarun@sarun-tuf: ~  
sarun@sarun-tuf:~$ type cd  
cd is a shell builtin  
sarun@sarun-tuf:~$ type cp  
cp is /usr/bin/cp  
sarun@sarun-tuf:~$ |
```

- If you want to grade cd program, we have to upgrade the entire shell
 - cd only does simple task → rarely be change, we always use cd. That's why cd is a shell builtin.
- show location of cp command → cp is external program.
 - when we use cp, OS load and run the cp program for us.

✓ System Calls

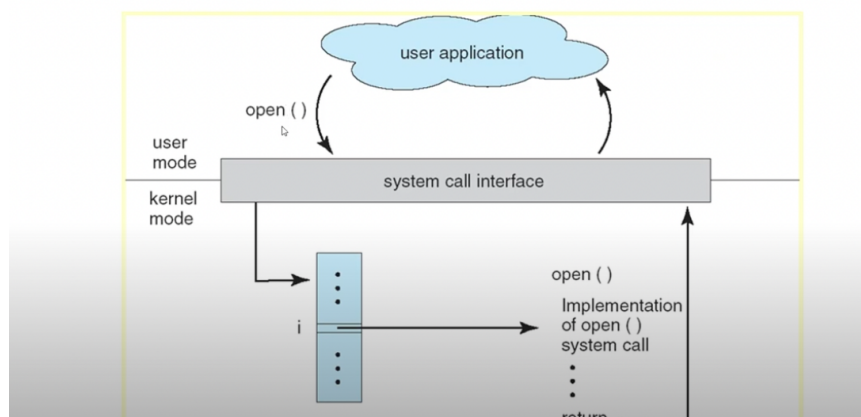
System Calls

- Programming interface to the services provided by the OS
 - Typically written in a high-level language (C or C++)
 - Mostly accessed by programs via a high-level Application Program Interface (API) rather than direct dependent OS system call use.
 - Three most common APIs are
 - Win32 API for Windows,
 - POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X), and
 - Java API for the Java virtual machine (JVM)
-
- Programming interface to the services provided by the OS.
 - you can think of them as function or library you can called.
 - Always written using high-level language (C, C++)
 - Actually, There are 2 levels of system calls
 - low-level → dependent on OS
 - if we write the program to use system call in this level → our program is less portable.
 - Less portable → Your code can't be easily port to be compile on some other OS platform, so that's why we our use high-level

- high-level
 - high-level API
 - 3 common API
 - Win32 API for Windows
 - POSIX API (we gonna learn this one) (may compatible with window based as well)
 - Java API for JVM

API - System Call - OS Relationship.(what happen when system call function is called)

API – System Call – OS Relationship



Assuming that, user program called open()

- open() → one of the system call

When a program call system call function → software interrupt occur.

- the way OS handle software interrupt is the same as hardware interrupt

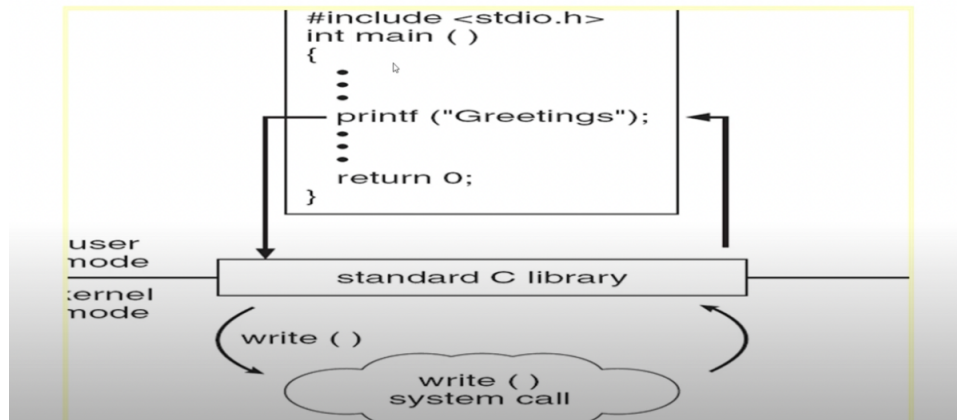
OS search the table to find the address in memory for the open() function.

- before it can run the function, it has to save the status of the current running program first.

👉 In summary, OS handled sw interrupt the same way as hw interrupt.

Standard C Library Example

Standard C Library Example



- The system call doesn't have to be called directly
- Many time, they are called indirectly from the function in the standard library in your language.
- For ex, we use printf function.
 - printf is use to write data to monitor, monitor is hardware
 - The user program can't access monitor directly, it has to ask OS to do the task.
 - Printf call write() (Assuming write is system call)
 - you do not call write() directly, it call from printf() that you call

Type of System Calls

Type of system calls depends on the services that OS provide.

- Process Control (focus)
 - it has system call that manage the process.
- File management
- Device management
- Information maintenance
- Communications (focus)

- Protection

System call can be group based on services that OS provide.

📌 Examples of Windows and Unix System Calls

Examples of Windows and Unix System Calls

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

Each OS has its own system call, if we don't use standard high-level API library.

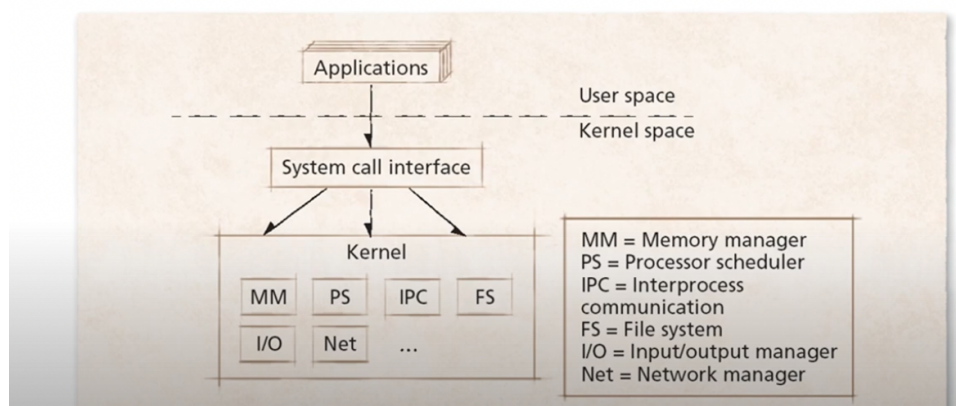
- The comparison between unix-based OS and Windows based OS.

✅ Kernel Architecture

Since OS in our class is kernel, As software developer, it's better to know how many way the kernel was designed and implemented. **ข้อดี ข้อเสีย ของแต่ละ approach**

📌 Monolithic Architecture

Monolithic Architecture



- Monolithic means big
- What does big mean?
 - You put every OS function in kernel



Advantage → Your OS gonna work very fast.

- when your program call system call, then your program go into kernel mode.
- And sometimes service that you called can't work alone (file system that need help of memory manager) → they are now in kernel mode → they can call directly here without switching back and forth between user mode and kernel mode.



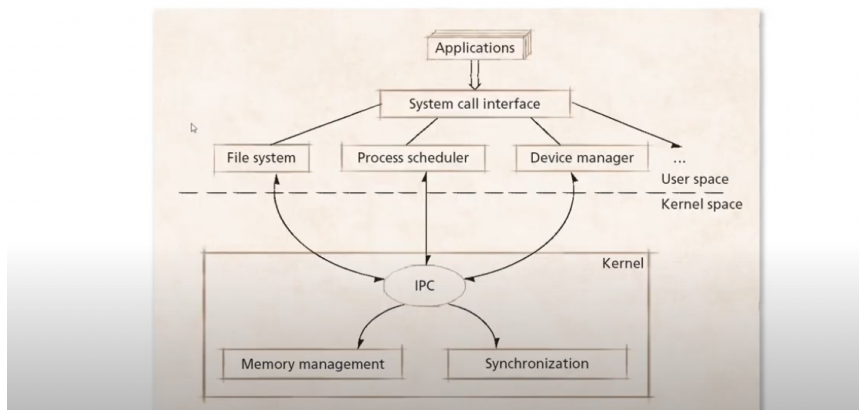
Disadvantage

- System is big → hard to modified (software developer)
 - If we modified one part, always effect some other part.
 - You have to test the whole system → after you modified ແຕ່ນິດ ເລີຍໆ ແບບ tiny part
- kernel is big → hard to port
 - If you want to port OS to run on some other hardware platform, It harder to port because your kernel is very big.



Microkernel Architecture

Microkernel Architecture



- This approach is opposite to Monolithic
 - it try to make the kernel as small as possible
 - by consider what are the essential task that must be in kernel
 - In this case → Memory management, Synchronization, IPC (inter process communication)
 - the rest of service is at user level
- When application call system call → call file system
 - first file system work in user mode, and if it need help from memory, the system must switch to kernel → done → switch back → need memory again → switch.



Advantage

- kernel is small → easy to modify and port to new hardware platform



Disadvantage

- slower than monolithic because switching back and forth

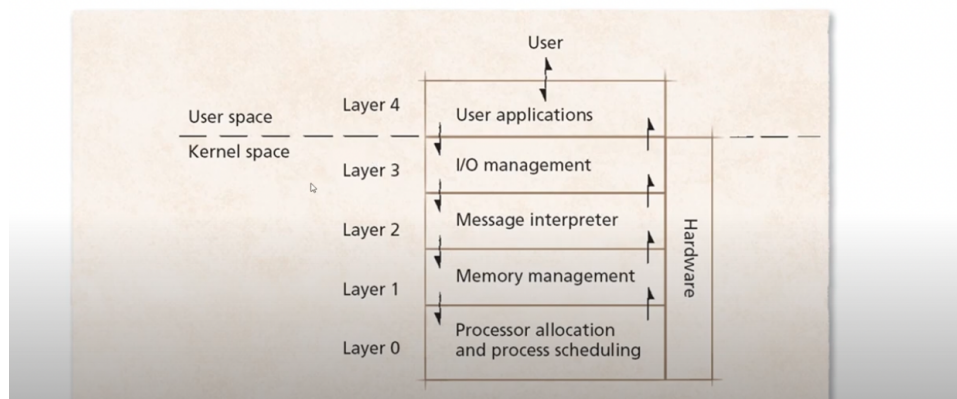
(almost all OS choose performance over modification and portability, so all essential OS task are in kernel, but to solve the problem of pure monolithic, the

software development approaches are used in the design and implementation of OS)

- 1 approach that can be used is Layered approach

Layered Architecture

Layered Architecture



- All OS task are still in kernel, but they are put in different layer
- The communication can occur only between layer ที่ติดกัน
 - if i/o need memory, i/o can access directly in memory. i/o passed request through message interpreter.
 - the system may work very slow.



Advantages

- if we have a very good design, layer boundary → when we modify something in 1 layer → it won't effect the rest of the system



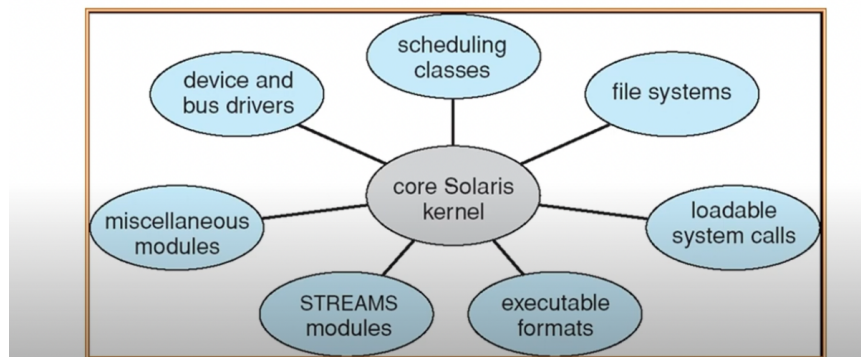
Disadvantages

- slow because of communication ไล่แค่ layer ที่ติดกัน

(The approach that we can use and that is better than layer architecture → Module Architecture)

Module Architecture (Solaris Example)

Module Architecture (Solaris Example)



- In this architecture, all OS essential tasks are in kernel, but there are developed in the form of module



Advantage

- If we design each module so well, then modification in 1 module won't affect the rest of the system
- (Solaris) It has a core kernel → you can think of it as center of the system
 - if module needs services from other module, they can use core kernel as the way to communicate, don't need to pass data to many layers

This module architecture allows us to use the idea of loadable module

- loadable module is the module that will be loaded when they are needed
 - when you turn your machine on, OS has to be loaded, so using this idea means instead of loading the entire OS → we'll load the OS with essential modules only, and then if OS faced a task that required additional modules → load them.
- The machine can be turned on faster → use less memory, load only part of OS

- The module that never be used, never be loaded